Introduction to Java Programming Language

Desenvolvimento de Software e Sistemas Móveis (DSSMV)
Licenciatura em Engenharia de Telecomunicações e Informática
LETI/ISEP

2026/27

Paulo Baltarejo Sousa
pbs@isep.ipp.pt

Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

1. Object-Oriented Programming Concepts
2. Features of the Java Programming Language
3. Java Basics
4. A Closer Look at Methods

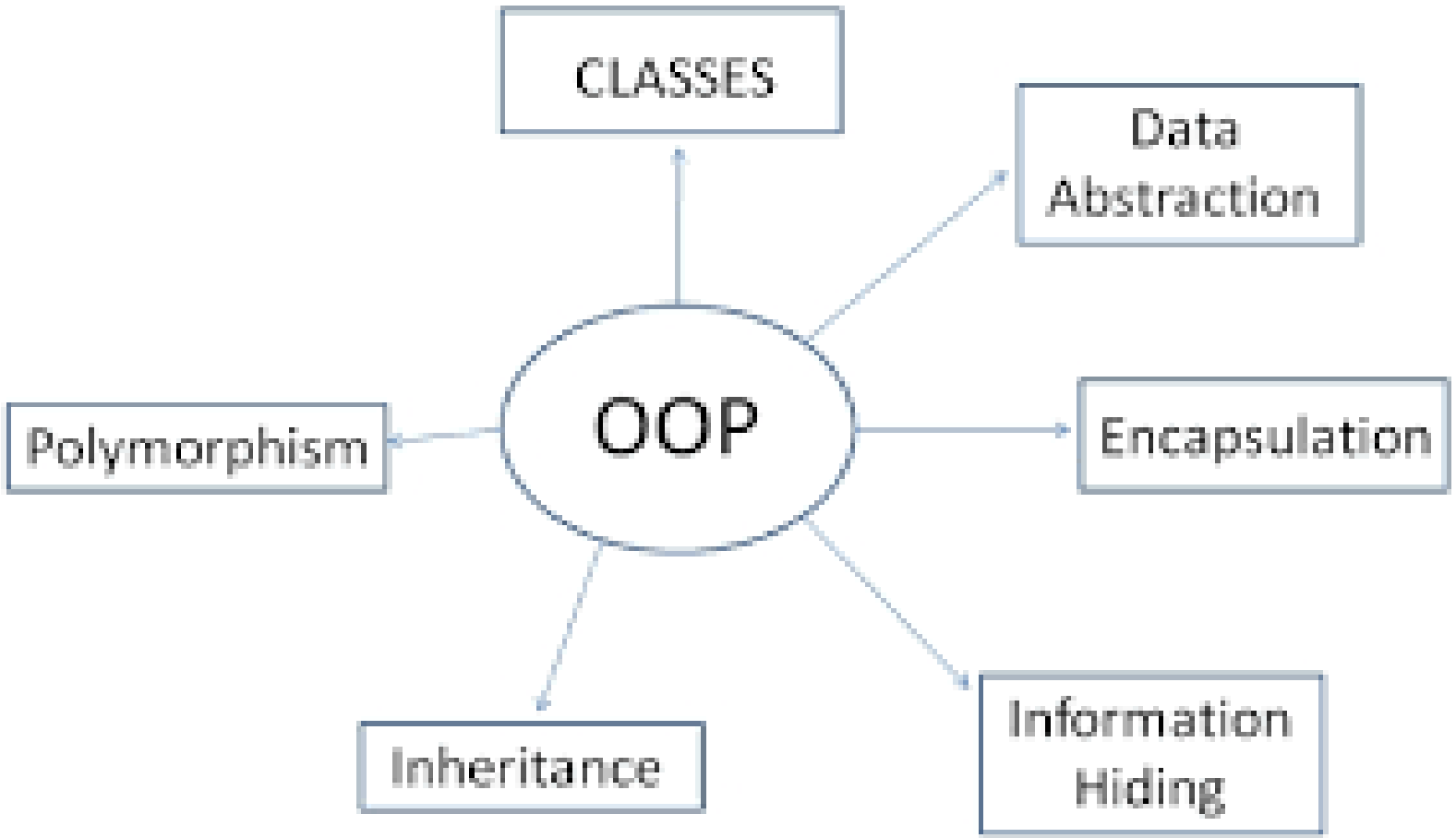
1 Object-Oriented Programming Concepts

2 Features of the Java Programming Language

3 Java Basics

4 A Closer Look at Methods

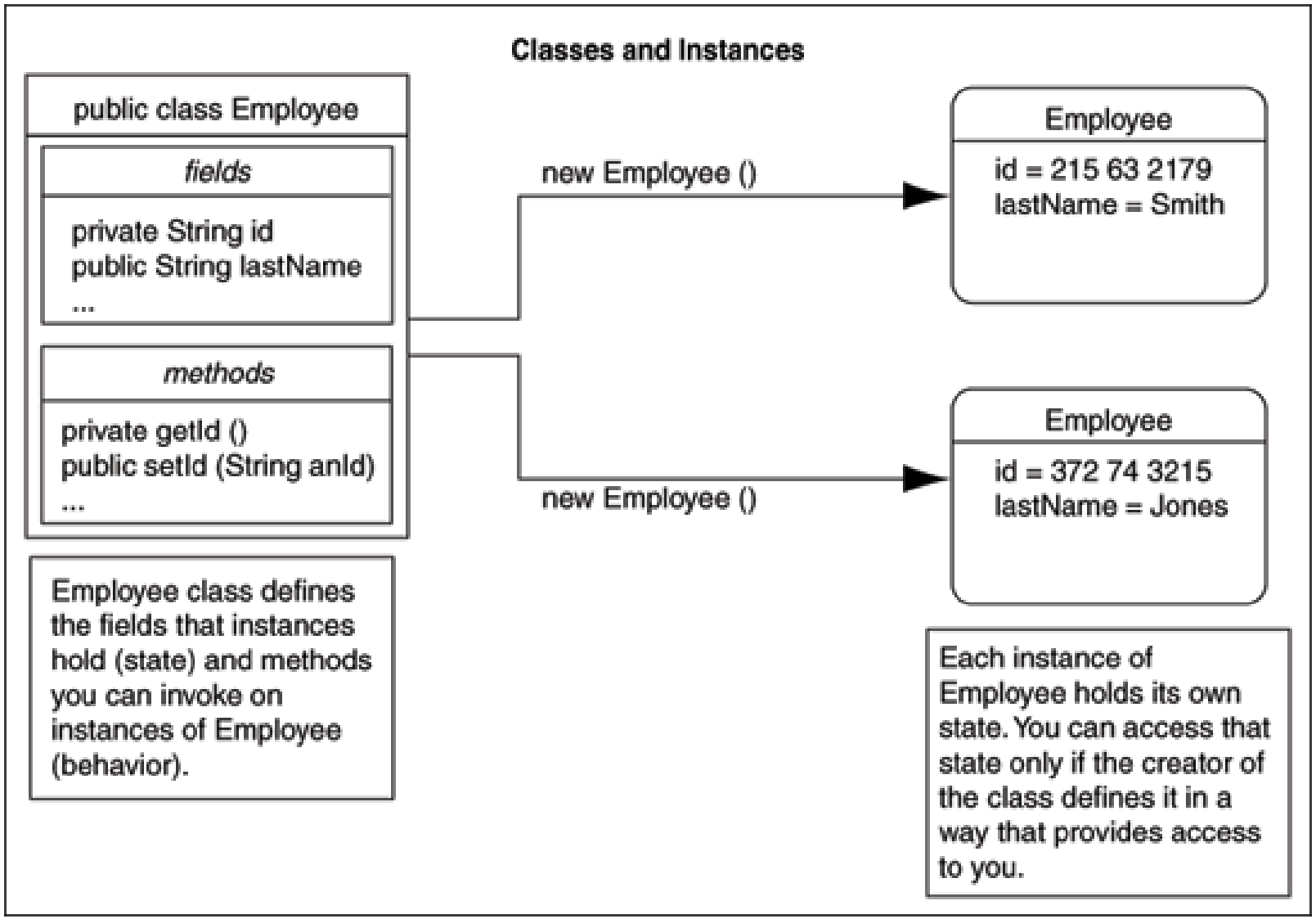
Object-Oriented Programming (OOP) (I)



OOP (II)

- Writing object-oriented programs involves creating **classes**, which are blueprints for **objects**;
- A class definition describes what **attributes** its objects will have and what those objects will be able to do.
- Attributes are the characteristics that define an object; they are **properties** of the object.
- An **object** is a specific, concrete **instance** of a class.
  - The values contained in an object's properties often differentiate instances of the same class from one another.
- A **method** is a self-contained block of program code that carries out some action.

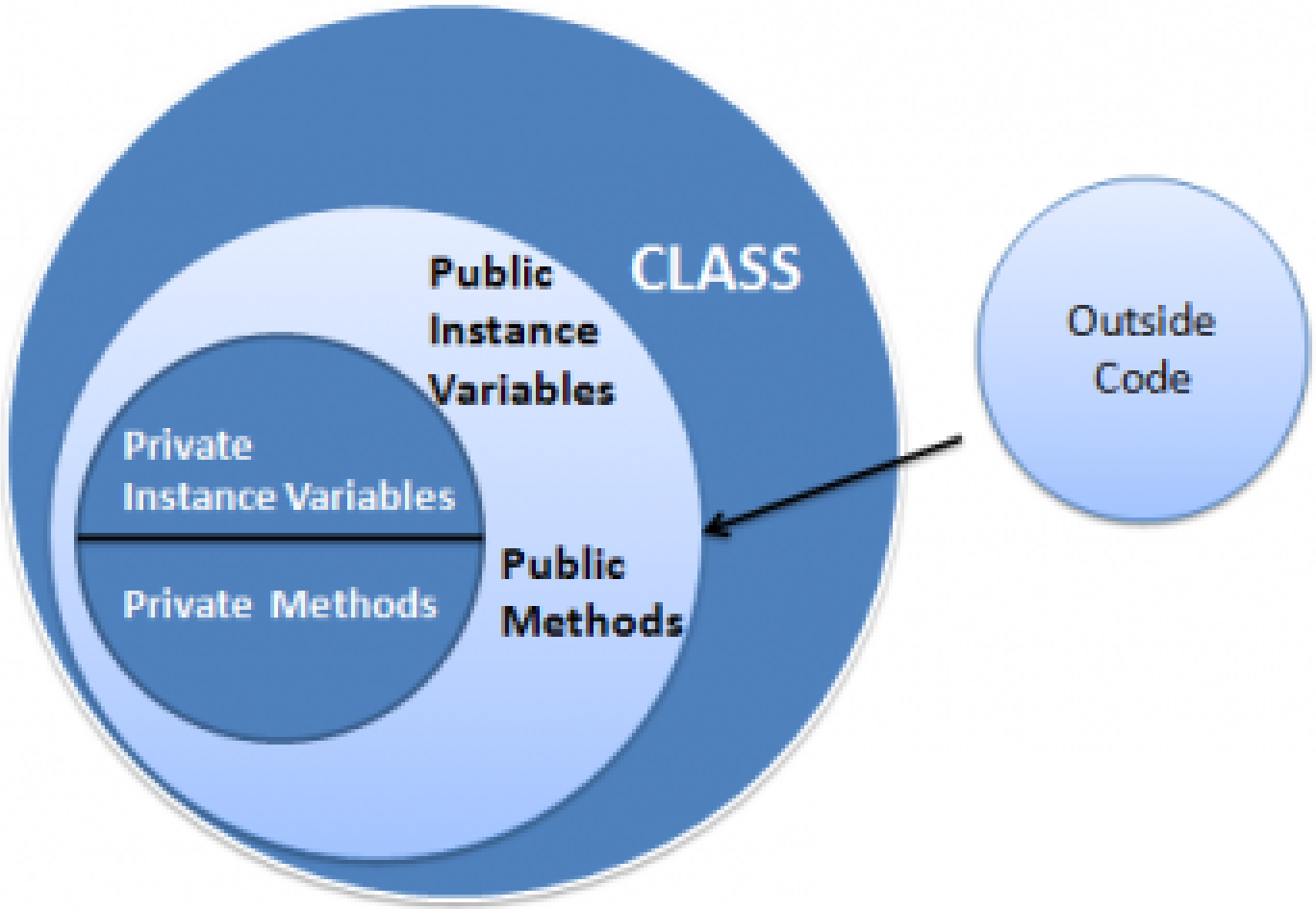
OOP: Classes and Objects



OOP: Encapsulation & Hiding (I)

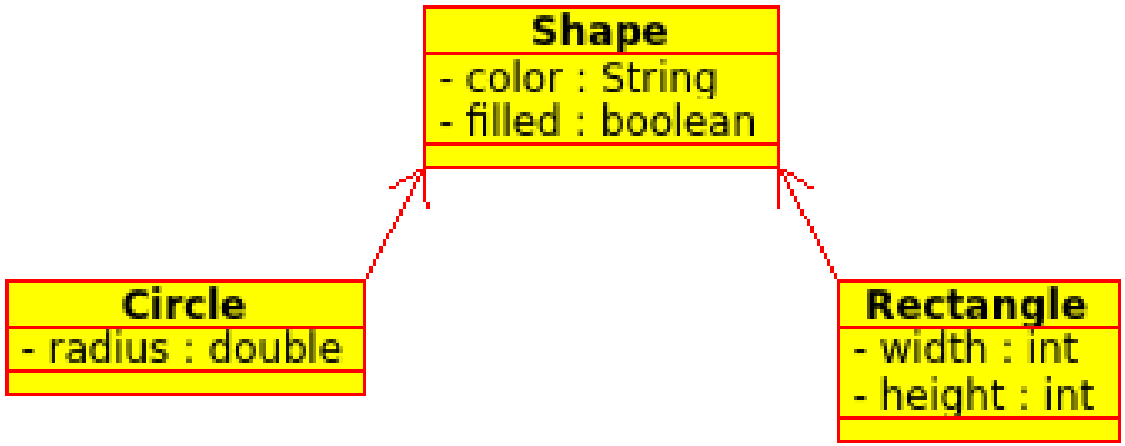
- In object-oriented classes, attributes and methods are **encapsulated** into objects.
  - It is the enclosure of data and methods within an object.
- Encapsulation allows you to treat all of an object's methods and data as a single entity.
- It also refers to the concealment (**hiding**) of an object's data and methods from outside sources.
  - Concealing data is sometimes called information hiding, and concealing how methods work is implementation hiding
  - It lets you hide specific object attributes and methods from outside sources and provides the security that keeps data and methods safe from inadvertent changes.

OOP: Encapsulation & Hiding (II)



OOP: Inheritance

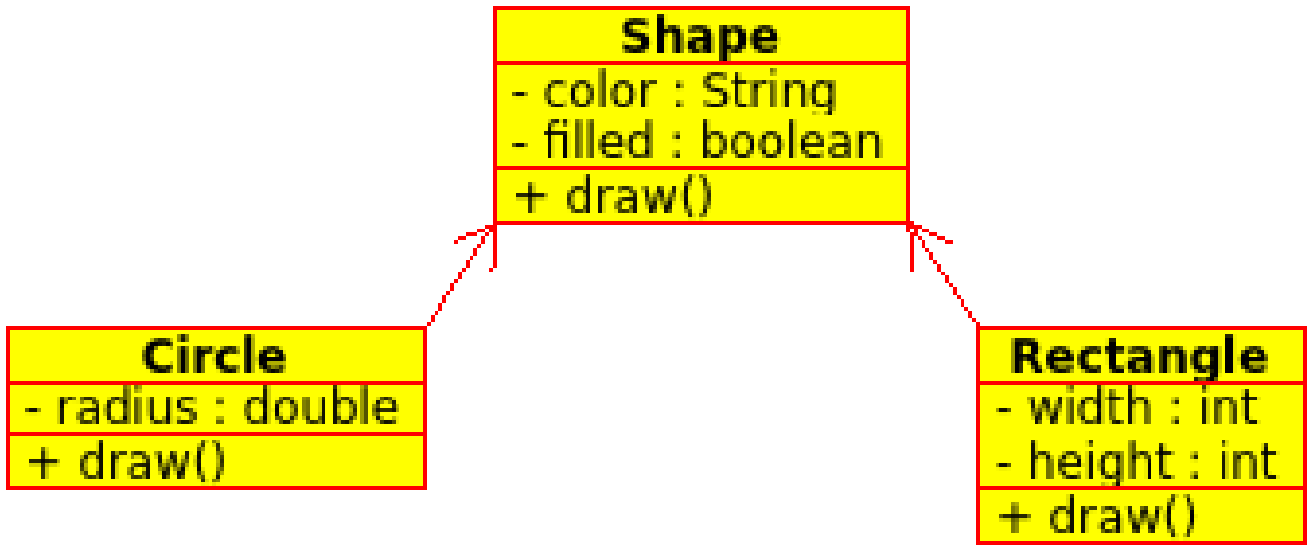
- **Inheritance** is the ability to create classes that share the attributes and methods of existing classes but with more specific features.



- When a **Class** extends another class it inherits all non-private members including fields and methods.
  - It can be best understood in terms of Parent and Child relationship, also known as **Super class** (Parent) and **Sub class** (child) in Java language.

OOP: Polymorphism

- **Polymorphism** means "many forms".
  - It could be static and dynamic both.
- **Overloading** is static polymorphism while, **overriding** is dynamic polymorphism.
  - Overloading in simple words means two methods having same method name but takes different input parameters.
  - Overriding means a derived class is implementing a method of its super class.



1 Object-Oriented Programming Concepts

2 Features of the Java Programming Language

3 Java Basics

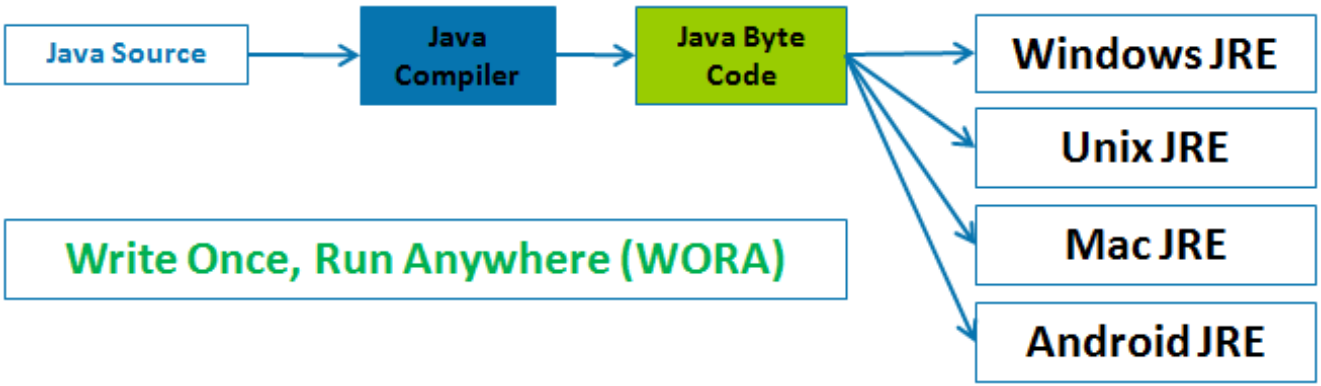
4 A Closer Look at Methods

Java (I)

- **Java** was developed by Sun Microsystems as an object-oriented language for general–purpose applications and for interactive, World Wide Web–based Internet applications.
- You can use Java to write a program that runs on any operating system (such as Windows, Mac OS, or Linux) or device (such as PCs, phones, and tablet computers).
  - Java can be run on a wide variety of computers and devices because it does not execute instructions on a computer directly. Instead, Java runs on a hypothetical computer known as the **Java Virtual Machine** (JVM).
- Java is a high-level programming language.
- The Java application source code is saved in a file; then, the Java compiler converts the source code into a binary program of **bytecode**.

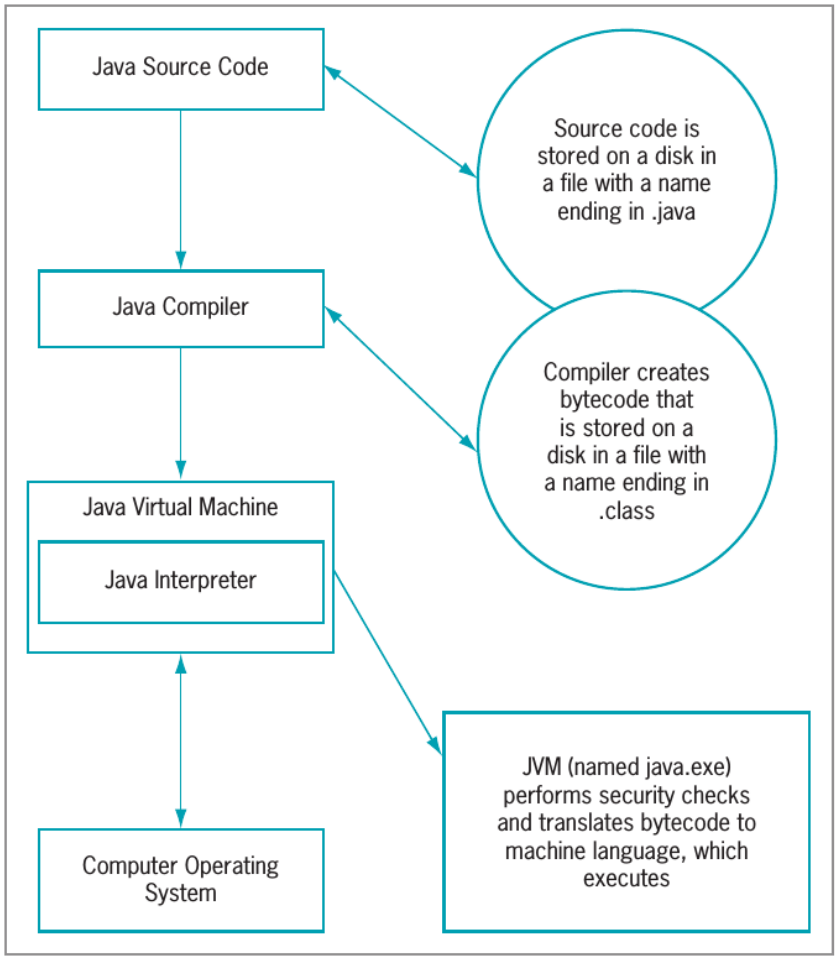
Java (II)

- **Write Once, Run Anywhere (WORA)** is a slogan developed by Sun Microsystems to describe the ability of one Java program version to work correctly on multiple platforms.



Java (III)

- A program called the **Java interpreter** then checks the **bytecode** and communicates with the operating system, executing the bytecode instructions line by line within the **Java Virtual Machine**.
  - Because the Java program is isolated from the operating system, the Java program also is isolated from the particular hardware on which it is run.



Java language (keywords)

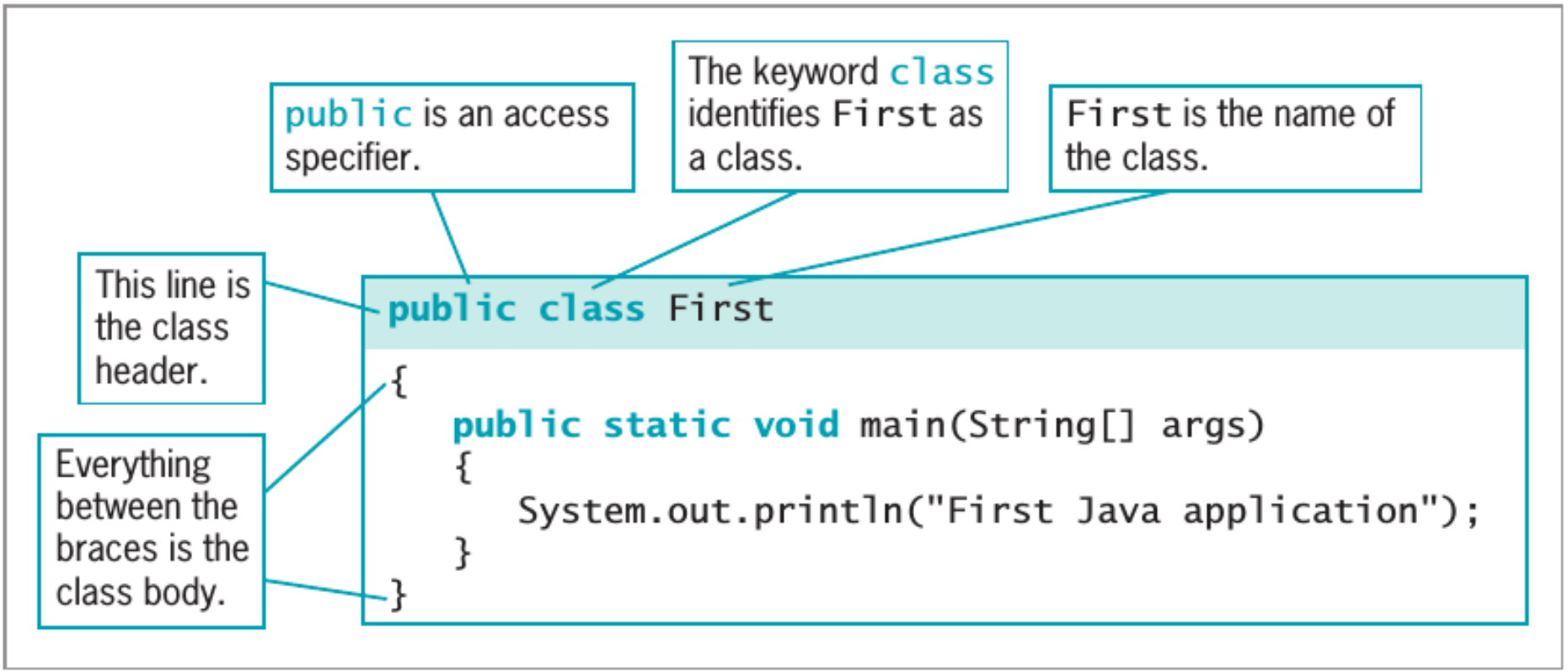
abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Simplest Java application (I)

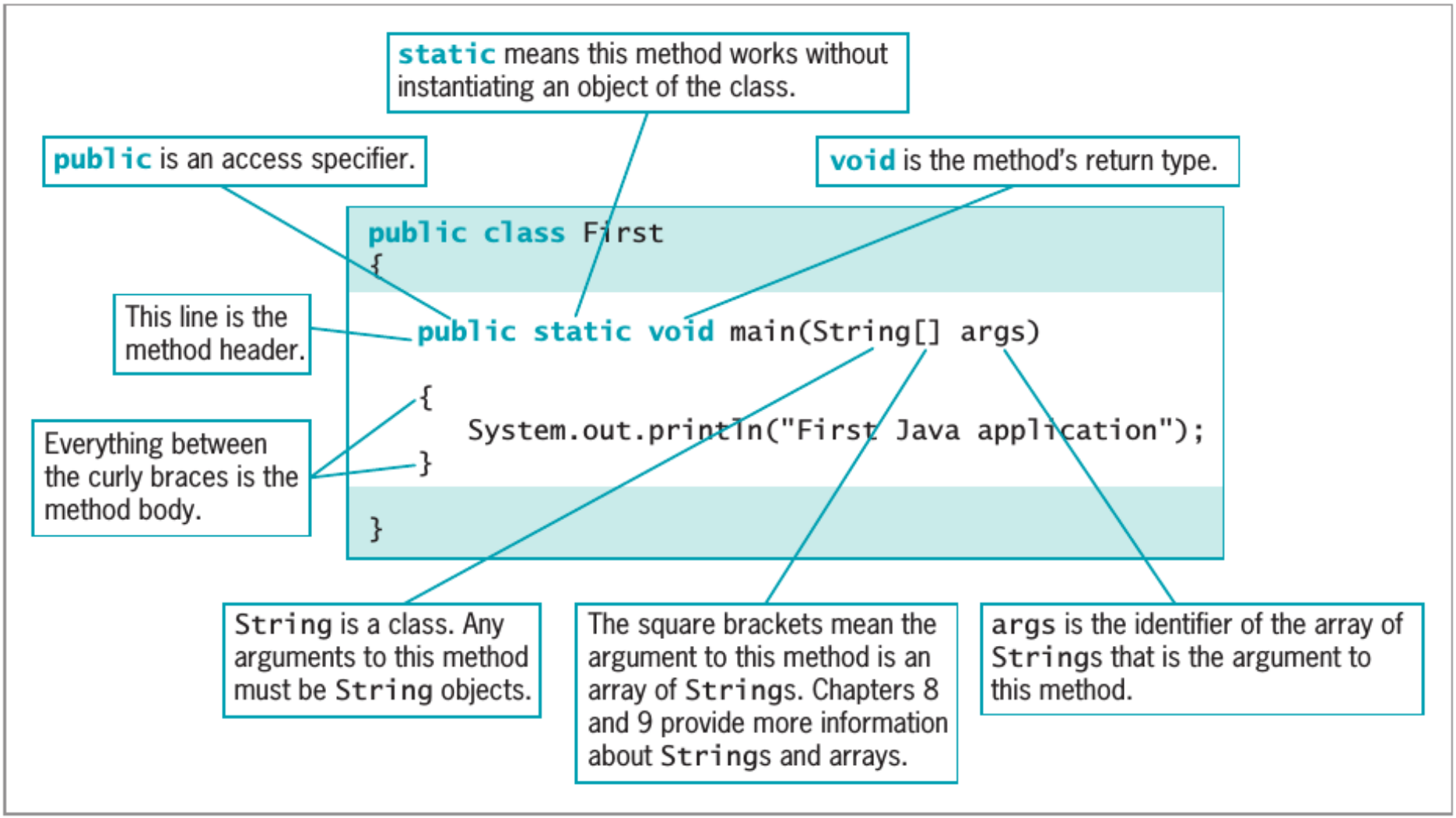
- Every program consists of one or more classes.
- **Every source file can contain at most one public class, and the name of the public class must match the name of the file containing the class**.
  - For example, the class `First` must be contained in the `First.java` file.

```
public class First
{
   public static void main(String[] args)
   {
      System.out.println("First Java application");
   }
}
```

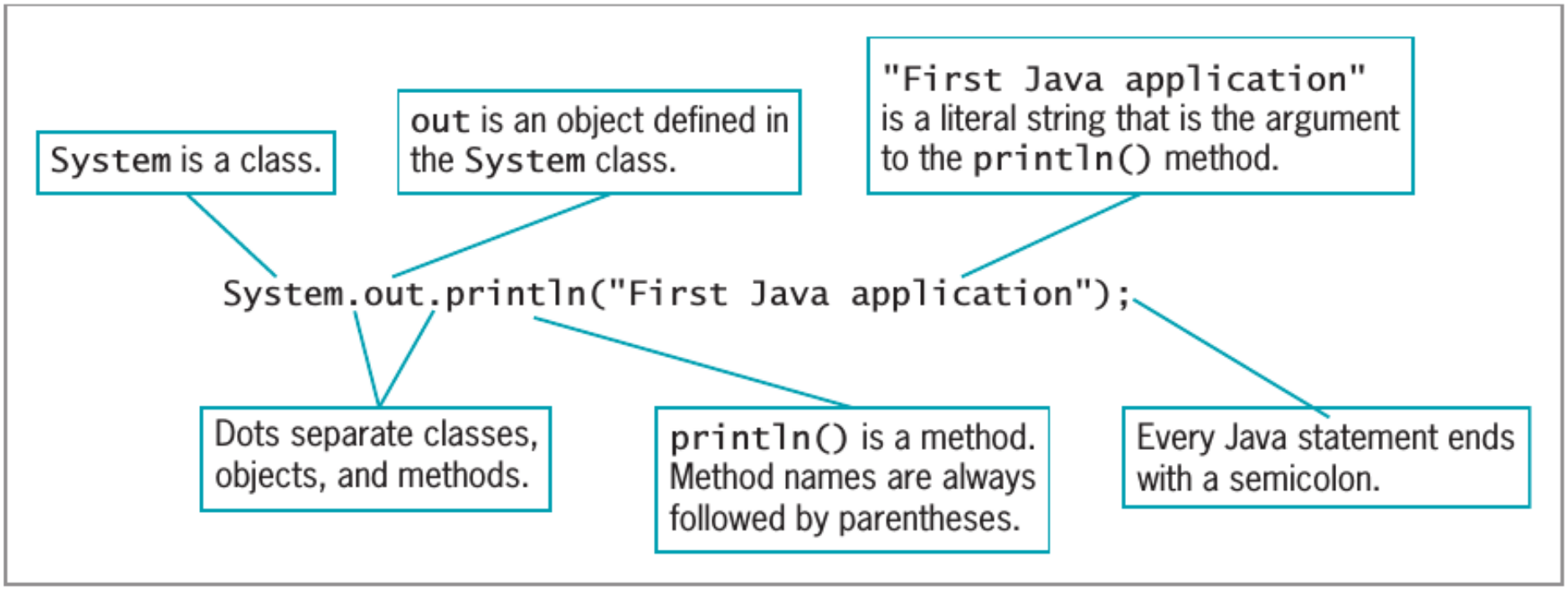
Simplest Java application (II)



Simplest Java application (III)



Simplest Java application (IV)



Compiling a Java Class

- To compile your source code from the command line, your prompt should show the folder or directory where your program file is stored.
- Then, you type `javac` followed by the name of the file that contains the source code.
- For example, to compile a file named `First.java`, you type the following and then press `Enter`:

```
> javac First.java
```

Running a Java Application

- To run the First application from the command line, you type the following:

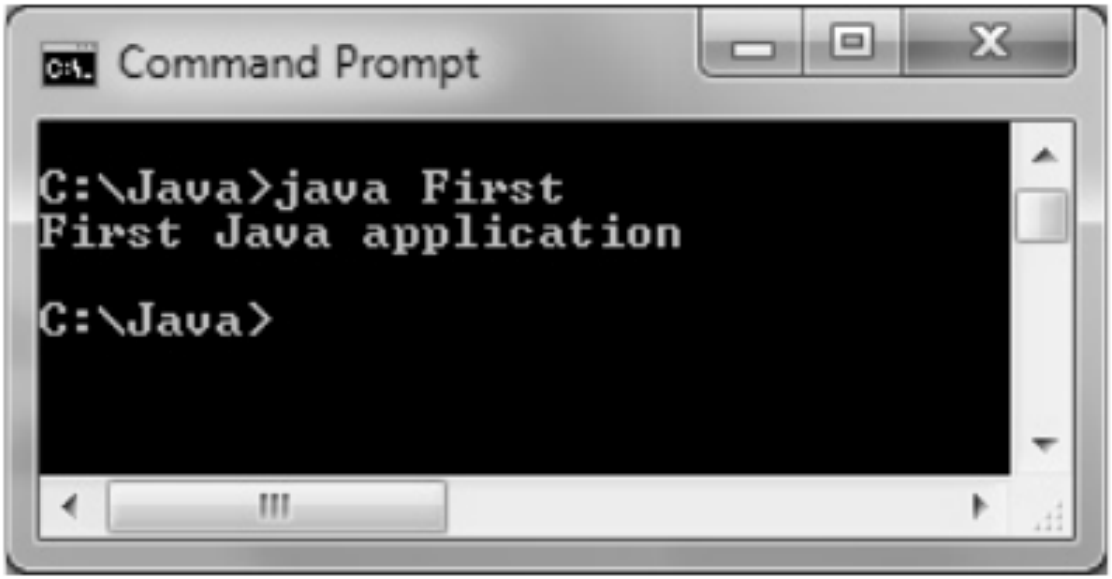
```
> java First
```

or

```
> java -cp .  First
```

or

```
> java -classpath .  First
```



1 Object-Oriented Programming Concepts

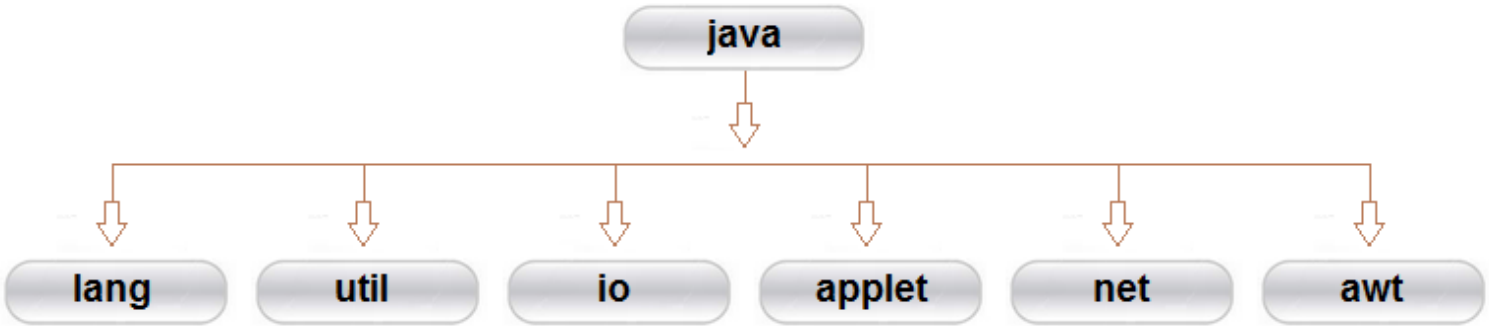
2 Features of the Java Programming Language

3 Java Basics

4 A Closer Look at Methods

What Is a Package?

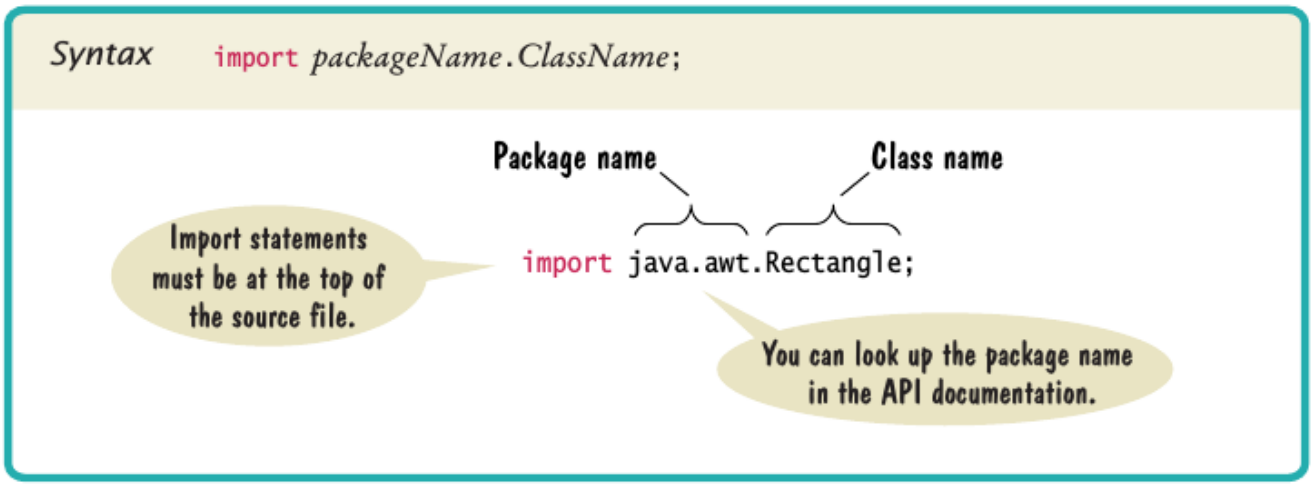
- A package is a **namespace for organizing classes and interfaces** in a logical manner.
  - A package is a collection of classes with a related purpose.
  - Java classes are grouped into packages. Use the `import` statement to use classes that are declared in other packages.
- Conceptually you can think of **packages as being similar to different folders on your computer**.
- Java library consists of several hundred packages.

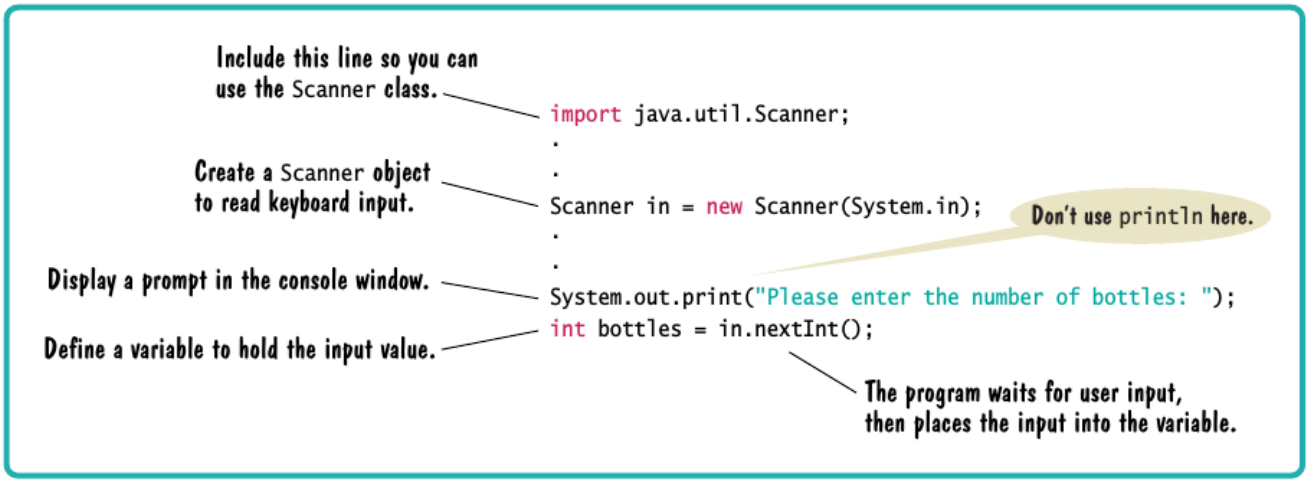


Using a Package

- Use the `import` statement to use classes that are declared in other packages.

Syntax `import packageName.ClassName;`





Class

- A Java application is composed by a set of classes.

Syntax *accessSpecifier* `class` *ClassName*

```
{
   instance variables
   constructors
   methods
}
```

```
public class Counter
{
   private int value;

   public Counter(int initialValue) { value = initialValue; }

   public void click() { value = value + 1; }
   public int getValue() { return value; }
}
```

Public interface

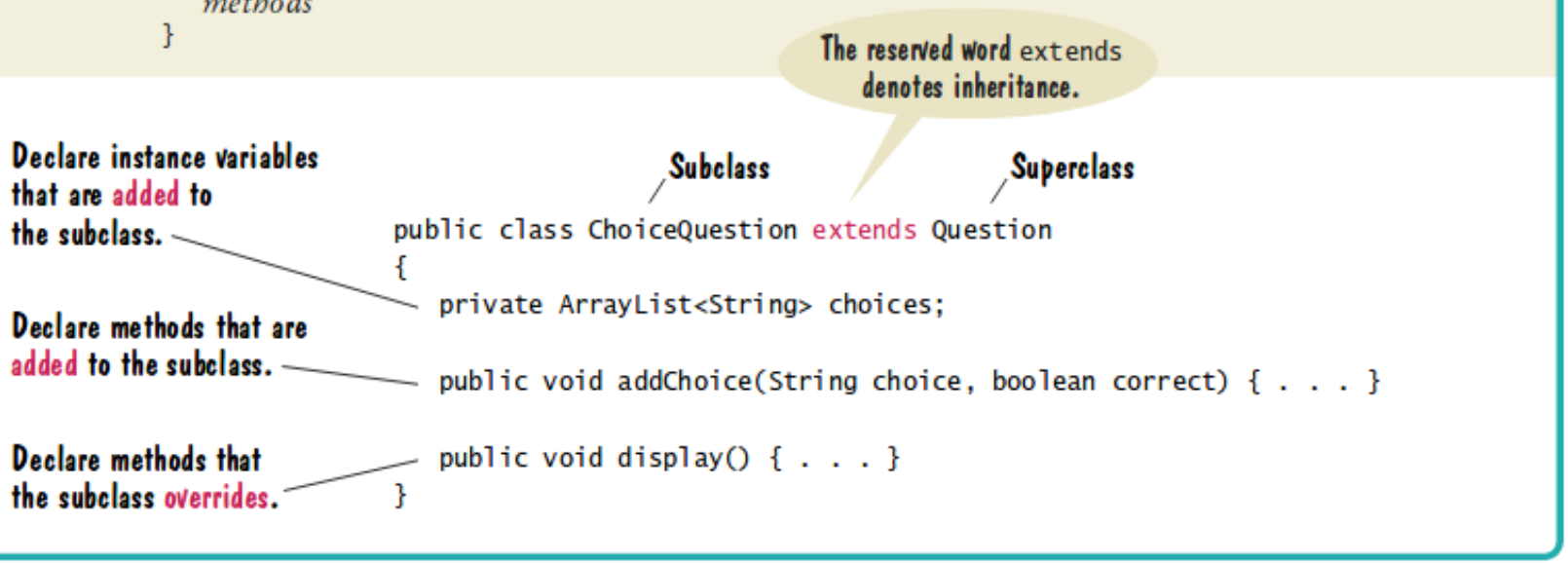
Private implementation

Subclass

- A class that inherits variables and methods from a superclass but may also add instance variables, add methods, or redefine methods.

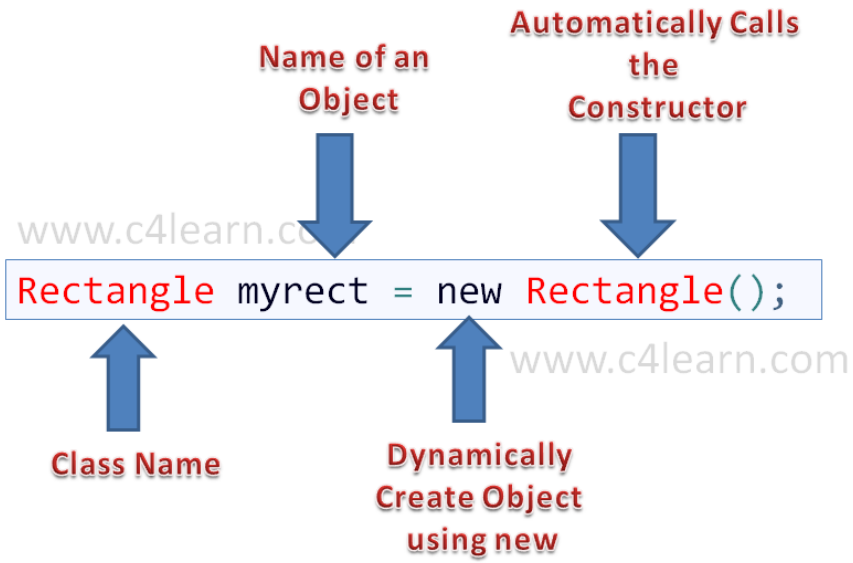
Syntax

```
public class SubclassName extends SuperclassName
{
   instance variables
   methods
}
```



What is an object?

- An object is an instance of a class.



What Is an Interface? (I)

- An **interface** is a contract between a class and the outside world.
  - When a class implements an interface, it promises to provide the behavior published by that interface.
- An **interface is a group of related methods with empty bodies** (abstract).

```
interface Bicycle {
  void changeCadence(int newValue);
  void changeGear(int newValue);
  void speedUp(int increment);
  void applyBrakes(int decrement);
}
```

- Implementing an interface allows a class to become more formal about the behavior it promises to provide.

What Is an Interface? (II)

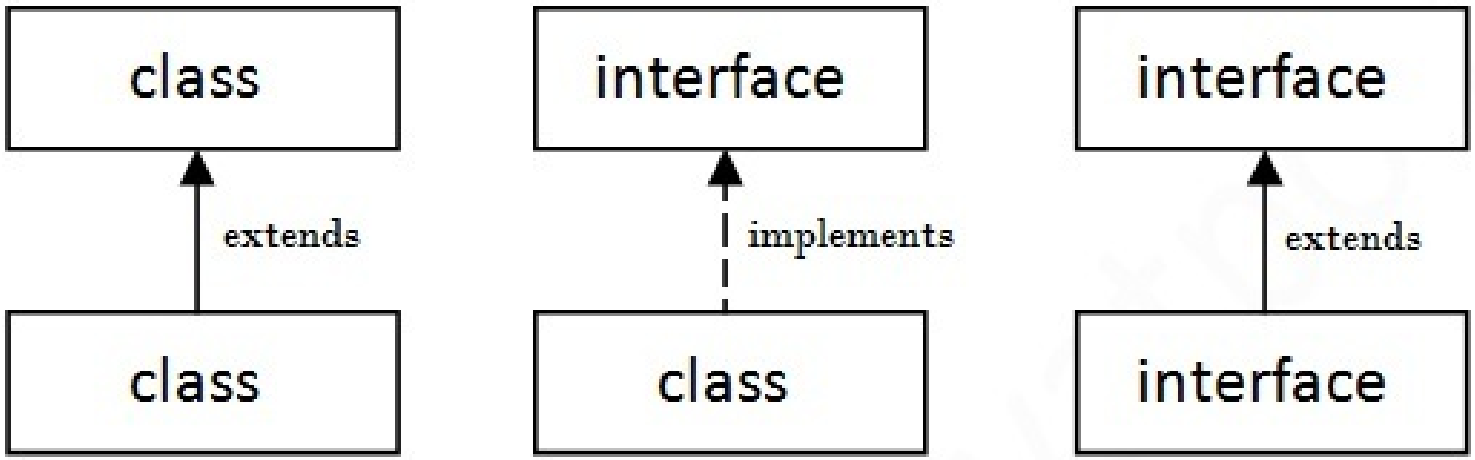
- If your class claims to implement an interface, all methods defined by that interface must appear in its source code before the class will successfully compile.

```
class MountainBike implements Bicycle {
  int cadence = 0;
  int speed = 0;
  int gear = 1;

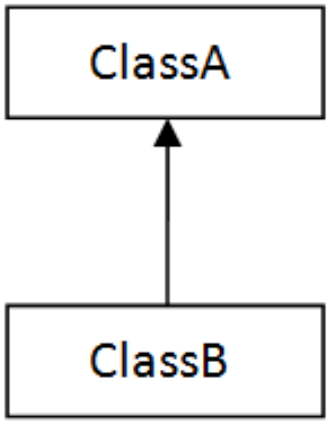
  void changeCadence(int newValue) {
    cadence = newValue;
  }
  void changeGear(int newValue) {
    gear = newValue;
  }
  void speedUp(int increment) {
    speed = speed + increment;
  }
  void applyBrakes(int decrement) {
    speed = speed - decrement;
  }
}
```

Classes and Interfaces

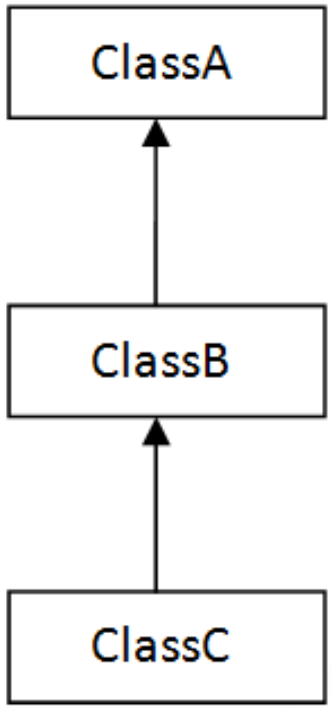
- A class **extends** another class, an interface **extends** another interface but a class **implements** an interface.



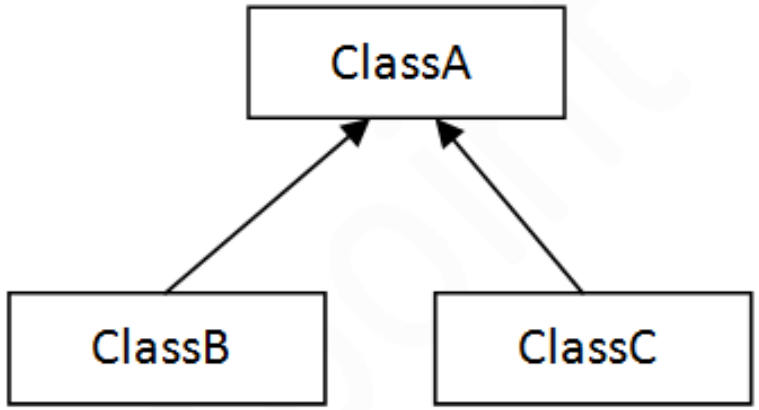
Types of inheritance (I)



1) Single



2) Multilevel



3) Hierarchical

- There can be three types of inheritance: single, multilevel and hierarchical.

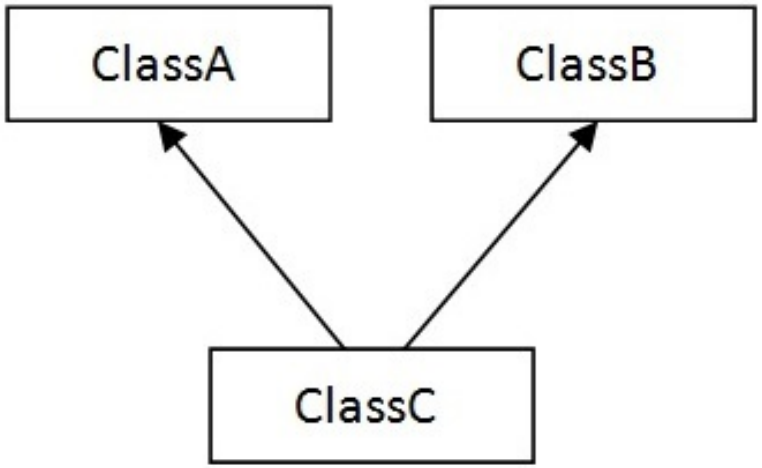
Types of inheritance (II)

```
class Animal{
  ...
}
class Dog extends Animal{
  ...
}
```

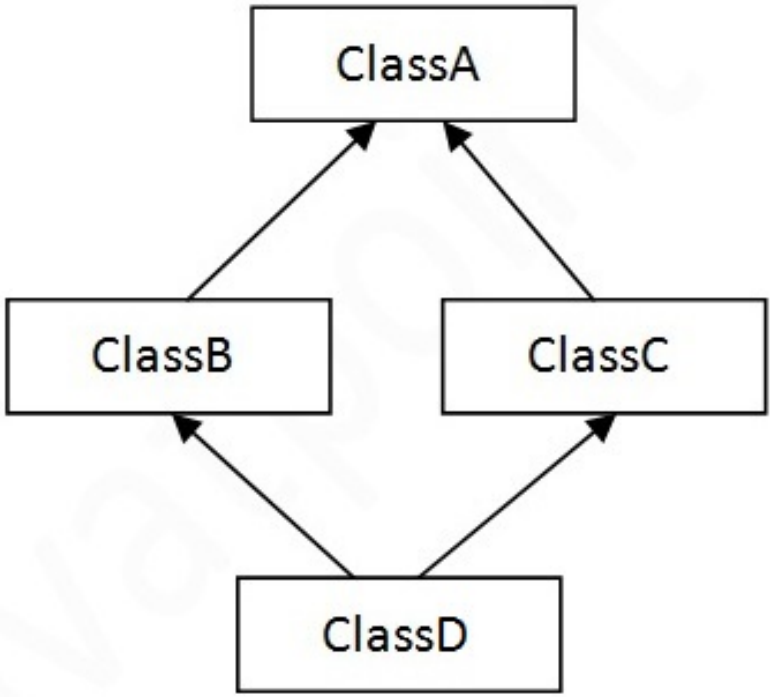
```
class Animal{
  ...
}
class Dog extends Animal{
  ...
}
class BabyDog extends Dog{
  ...
}
```

```
class Animal{
  ...
}
class Dog extends Animal{
  ...
}
class Cat extends Animal{
  ...
}
```

Types of inheritance (III)



4) Multiple



5) Hybrid

- Multiple and hybrid inheritance is supported through **interface** only

Types of inheritance (IV)

```
interface A {
  public void methodA();
}
interface B {
  public void methodB();
}
class MultipleInheritance implements A,B{
  public void methodA() {
    System.out.println("Calling methodA");
  }
  public void methodB() {
    System.out.println("Calling methodB");
  }
}
```

```
interface A {
  public void methodA();
}
interface B extends A {
  public void methodB();
}
interface C extends A {
  public void methodC();
}
class HybridInheritance implements B, C {
  public void methodA() {
    System.out.println("Calling methodA");
  }
  public void methodB() {
    System.out.println("Calling methodB");
  }
  public void methodC() {
    System.out.println("Calling methodC");
  }
}
```

1 Object-Oriented Programming Concepts

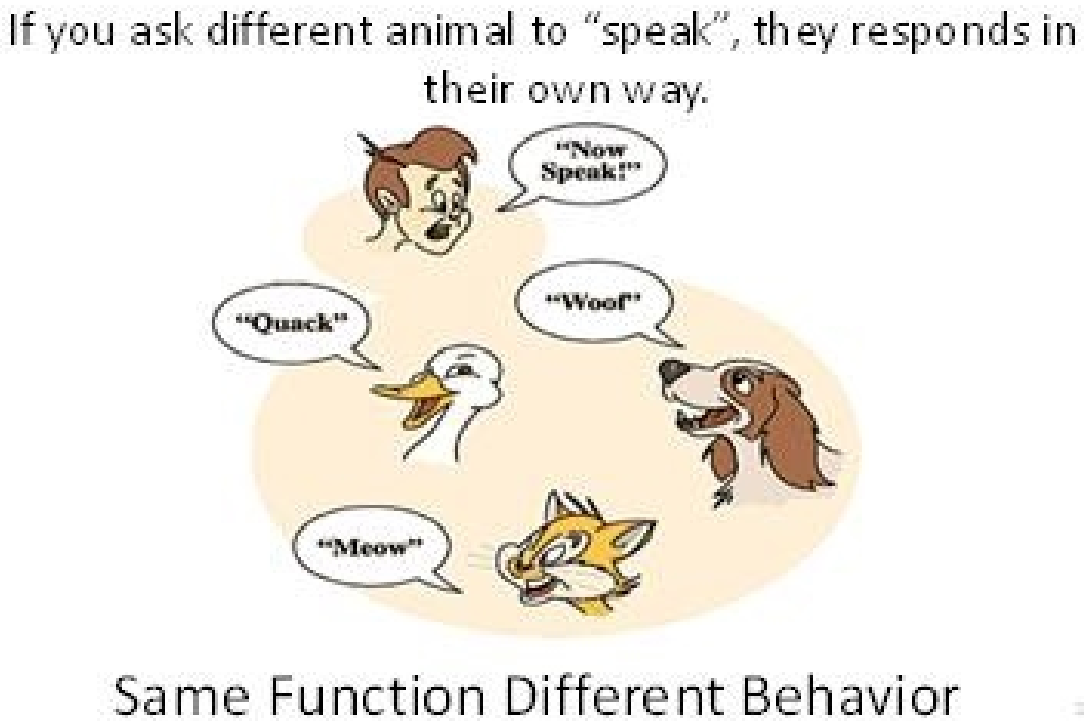
2 Features of the Java Programming Language

3 Java Basics

4 A Closer Look at Methods

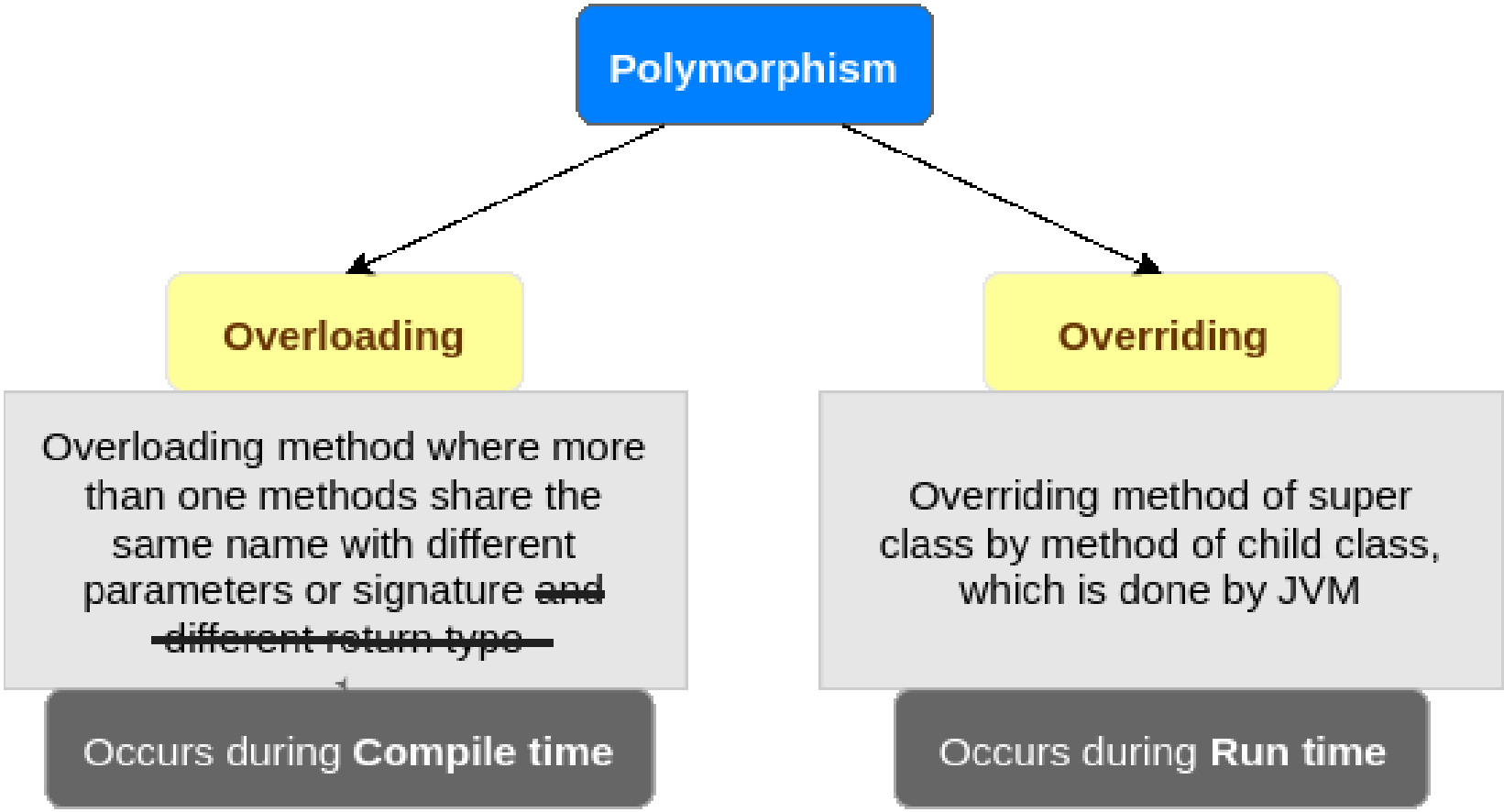
Polymorphism (I)

- The word “Polymorphism” comes from two Greek words, “many” and “form”.
  - We can illustrate the idea of polymorphism easily using the scenario where different animals are asked to “speak”.



Polymorphism (II)

- **Overloading** is static polymorphism.
- **Overriding** is dynamic polymorphism.



Overloading Methods (I)

- In Java it is possible **to define two or more methods within the same class that share the same name**, as long as their parameter declarations are different.
  - When this is the case, the **methods are said to be overloaded**, and the process is referred to as method overloading.
- Method overloading is one of the ways that Java supports **polymorphism**.
- When an overloaded method is invoked, Java uses the **type** and/or **number of arguments** as its guide to determine which version of the overloaded method to actually call.
  - Thus, overloaded methods must differ in the type and/or number of their parameters.
- While overloaded methods may have different **return types**, the return type alone **is insufficient to distinguish two versions of a method**.

Overloading Methods (II)

```
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }
  // Overload test for one integer parameter.
  void test(int a) {
    System.out.println("a: " + a);
  }
  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }
  // overload test for a double parameter
  double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
  }
}
```

Overloading Constructors

```
class Box {
  double width;
  double height;
  double depth;
  // constructor used when all dimensions specified
  Box(double w, double h, double d) {
    this.width = w;
    this.height = h;
    this.depth = d;
  }
  // constructor used when no dimensions specified
  // use -1 to indicate an uninitialized box
  Box() {
    this.width = -1;
    this.height = -1;
    this.depth = -1;
  }
  // constructor used when cube is created
  Box(double len) {
    this.width = this.height = this.depth = len;
  }
}
```

Overriding

```
public class ABC{
  public void myMethod(){
    System.out.println("Overridden Method: ABC");
  }
}
```

```
public class XYZ extends ABC{
  public void myMethod(){
    System.out.println("Overriding Method: XYZ");
  }
}
```

```
public class Main{
  public static void main(String args[]){
    ABC obj1 = new ABC();
    obj1.myMethod(); // This would call the myMethod() of parent class ABC
    XYZ obj2 = new XYZ();
    obj2.myMethod(); // This would call the myMethod() of child class XYZ
    ABC obj3 = new XYZ();
    obj3.myMethod(); // This would call the myMethod() of child class XYZ
  }
}
```

Using Objects as Parameters

- Methods and constructors

```
class Box {
  double width;
  double height;
  double depth;
  // This is constructor for Box that receive an object.
  Box(Box obj) {
    width = obj.width;
    height = obj.height;
    depth = obj.depth;
  }
  // return true if o is equal to the invoking object
  boolean equals(Box obj) {
    if( this. width == obj.width && this.height == obj.height && this.depth == obj.
          depth)
    return true;
    else
    return false;
  }
}
```

A Closer Look at Argument Passing

- In general, there are two ways that a computer language can pass an argument to a method:
  - The first way is **call-by-value**.
    - This approach copies the value of an argument into the formal parameter of the method.
    - Therefore, **changes made to the parameter of the method have no effect on the argument**.
  - The second way an argument can be passed is **call-by-reference**.
    - In this approach, a reference to an argument (not the value of the argument) is passed to the parameter. Inside the method, this reference is used to access the actual argument specified in the call.
    - This means that **changes made to the parameter will affect the argument used to call the method**.
- In Java
  - When you pass a **primitive type** to a method, **it is passed by value**.
  - When you pass an **object** to a method, **it is passed by reference**.

A Closer Look at Returning Types (I)

- **A method can return any type of data**, including class and primitive types as well as `void`.

```
public class Box {
  private double width;
  private double height;
  private double depth;
  public Box(double width, double height, double depth) {
    this.width = width;
    this.height = height;
    this.depth = depth;
  }
  public double getWidth() {
    return width;
  }
  public void setWidth(double width) {
    this.width = width;
  }
  public double getVolume() {
    return (width * height * depth);
  }
  public Box getBoxObject() {
    Box box = new Box(10,10, 10);
    return box;
  }
}
```

A Closer Look at Returning Types (II)

- Returning a **value**

```
public class Box {
  ...
  public double getWidth() {
    return width;
  }
  ...
}
```

- Returns an object **reference**
  - All objects are dynamically allocated in the **heap** using `new` operator.
    - The object will continue to exist as long as there is a reference to it somewhere in your program.

```
public class Box {
  ...
  public Box getBoxObject() {
    Box box = new Box(10,10, 10);
    return box;
  }
}
```